

COMPARATIVE ANALYSIS OF DEEP LEARNING ARCHITECTURES FOR SENTIMENT ANALYSIS OF MOVIE REVIEWS

Interim report on
Deep Learning Project [ICT-4442]

Submitted By

D. Vishwatej (230953220)
Abhay Pratap Singh (230911522)
Rallapalli Dheeraj Chowdary (230911530)
Y Kedarnath Chowdary (230911176)



**SCHOOL OF COMPUTER ENGINEERING
MANIPAL INSTITUTE OF TECHNOLOGY,
MANIPAL ACADEMY OF HIGHER EDUCATION**

October 2026

PART B : INTERIM REPORT

Phase 2

Team No. and Names (with Registration Numbers):

1. D. Vishwatej (230953220)
2. Abhay Pratap Singh (230911522)
3. Rallapalli Dheeraj Chowdary (230911530)
4. Y Kedarnath Chowdary (230911176)

Title of the Project :

Comparative Analysis of Deep Learning Architectures for Sentiment Analysis of Movie Reviews

GitHub Repository Link:

<https://github.com/ARTEMISFOWL-01/ICT4442-DL-Project.git>

1. LITERATURE REVIEW

Sentiment classification on movie reviews has been studied with a range of architectures over the past decade, from simple bag-of-words models to large pretrained Transformers. The ten papers below cover the techniques directly relevant to the four models we are comparing. Table I groups them by method, dataset, and their connection to our work.

Maas et al. [1] published the IMDb review corpus that we use and showed that learning word vectors jointly with a sentiment objective outperforms purely unsupervised embeddings. Kim [2] later showed that applying multiple 1D convolution kernels of different widths to word embeddings can match or beat much deeper networks on several sentence-classification benchmarks, which directly informed our CNN design. Ali et al. [3] carried out a comparison of CNN, LSTM, and hybrid architectures on IMDb reviews, confirming that the dataset is a useful testbed for multi-model studies like ours.

On the recurrent side, Hochreiter and Schmidhuber [4] introduced the LSTM cell to deal with vanishing gradients in long sequences, and Schuster and Paliwal [5] extended recurrent networks to read input in both directions. Together, these two ideas underpin the BiLSTM architecture we train. For Transformers, Vaswani et al. [8] replaced recurrence entirely with self-attention, Devlin et al. [6] showed that pretraining a deep bidirectional Transformer (BERT) on large corpora gives strong results across many NLP tasks, and Sanh et al. [7] distilled BERT into a smaller model (DistilBERT) that keeps about 97% of BERT's accuracy at 40% fewer parameters. Yang et al. [9] and Howard and Ruder [10] further explored attention-based document classification and language-model fine-tuning respectively, both of which support the transfer-learning approach behind our DistilBERT setup.

TABLE 1. Literature Review Summary

Paper (Author, Year)	Method	Dataset	Key Result	Relevance to Project
Maas et al. (2011) [1]	Sentiment-aware word vectors	IMDb (50k)	Introduced IMDb benchmark; joint unsupervised–supervised features boost sentiment accuracy	Source of the dataset; shows value of learning task-specific word representations
Kim (2014) [2]	1D-CNN for sentence classification	SST-2, MR, TREC	Multi-kernel CNN matches deeper models on several benchmarks	Basis for the multi-kernel (3, 4, 5) CNN we implement
Ali et al. (2019) [3]	Deep learning models for movie-review sentiment	IMDb	Compared CNN, LSTM, and hybrid architectures on IMDb	Closest prior multi-model comparison on the same dataset
Hochreiter & Schmidhuber (1997) [4]	LSTM	Synthetic sequences	Gated memory cells fix the vanishing gradient problem for long sequences	Core cell used inside the BiLSTM we train
Schuster & Paliwal (1997) [5]	Bidirectional RNN	TIMIT phoneme	Reading sequences in both directions gives better representations	Reason we run the LSTM in both directions
Devlin et al. (2019) [6]	BERT	GLUE, SQuAD	Pretraining a deep bidirectional Transformer beats task-specific models on 11 benchmarks	Parent model from which DistilBERT is distilled
Sanh et al. (2019) [7]	DistilBERT	GLUE	Keeps ~97% of BERT accuracy; 40% smaller, 60% faster	The specific pretrained checkpoint we fine-tune
Vaswani et al. (2017) [8]	Transformer (self-attention)	WMT translation	Self-attention removes the need for recurrence or convolution	Architecture behind all BERT-family models
Yang et al. (2016) [9]	Hierarchical Attention Networks	Yelp, Yahoo, Amazon, IMDb	Word- and sentence-level attention improves document classification	Shows that attention helps for review-length documents
Howard & Ruder (2018) [10]	ULMFiT	IMDb, TREC, AG, DBpedia	Fine-tuning a pretrained language model matches or beats task-specific training	Supports the transfer-learning strategy behind DistilBERT fine-tuning

2. DATASET ACQUISITION & PREPROCESSING

Dataset

We use the **IMDb Large Movie Review Dataset** released by Maas et al. [1], which can be downloaded from <https://ai.stanford.edu/~amaas/data/sentiment/>. The corpus contains 50,000 movie reviews, split evenly between 25,000 positive and 25,000 negative labels. The official release already separates 25,000 reviews for training and 25,000 for testing, plus 50,000 unlabelled reviews that we do not use. Because the class distribution is perfectly balanced (50/50), accuracy is a meaningful metric without class-weighting adjustments.

Data Splits

We partitioned the official 25,000-review training set into a smaller training portion and a validation set using `sklearn.model_selection.train_test_split` with `test_size=5000`, `random_state=42`, and `stratification` on labels:

- **Training set:** 20,000 samples (10,000 positive, 10,000 negative)
- **Validation set:** 5,000 samples (2,500 positive, 2,500 negative)
- **Test set:** 25,000 samples (12,500 positive, 12,500 negative) — official test split, kept completely separate

The test set is never touched during training or hyperparameter tuning. All model-selection decisions (early stopping, learning-rate reduction, checkpoint saving) depend only on the validation loss.

Preprocessing Pipeline

The three scratch-trained models (MLP, 1D-CNN, BiLSTM) share the same Keras tokenizer and padding setup:

1. **Tokenization:** We fit a Keras `Tokenizer` on the 20,000 training reviews only, with a vocabulary cap of 20,000 words and an out-of-vocabulary token (`<OOV>`). The fitted tokenizer is saved to `shared/tokenizer.pkl` and loaded by all three notebooks, so every model sees exactly the same integer-encoded input. Fitting on training data alone prevents any information from validation or test reviews from leaking into the vocabulary.
2. **Sequence encoding:** Raw text is converted to integer sequences using `texts_to_sequences()`.
3. **Padding and truncation:** Sequences are post-padded and post-truncated to `MAX_LEN=512` tokens. A histogram of training-set review lengths (Figure 1) shows that 92.3% of reviews fit within 512 words, so most reviews are padded rather than cut.
4. **Label encoding:** Labels are binary integers (0 for negative, 1 for positive).

DistilBERT preprocessing differs in two ways.

First, it uses its own pretrained WordPiece tokenizer (`DistilBertTokenizerFast` from `distilbert-base-uncased`) instead of the shared word-level tokenizer, because subword tokenization was part of its pretraining and mixing tokenizers would hurt performance. Second, the maximum sequence length is 256 subword tokens rather than 512, because the Transformer’s quadratic attention memory makes 512 tokens infeasible on a 16 GB Tesla T4. Raw review text is kept as-is, without lowercasing, stemming, or punctuation removal, since the pretrained model already expects natural text.

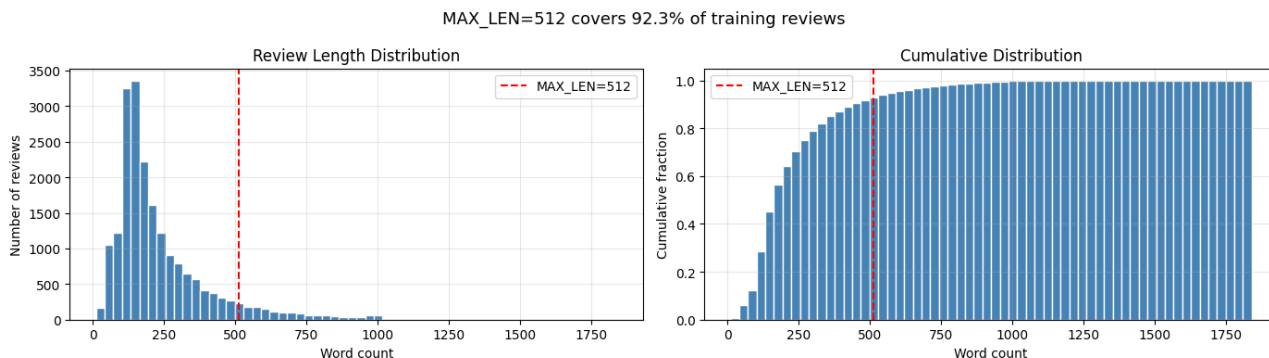


Figure 1: Review-length distribution and cumulative coverage for `MAX_LEN=512`; 92.3% of training reviews fit without truncation.

3. MODELS IMPLEMENTED SO FAR

All four proposed models have been implemented, trained, and evaluated on the validation set. Each architecture was chosen to represent a different family of deep learning approaches to text classification: a non-sequential baseline, a local-pattern detector, a sequential model with long-range memory, and a pretrained contextual model. Below we describe each one along with its training behaviour.

A. Multi-Layer Perceptron (MLP)

The MLP is the simplest model in our comparison and acts as a bag-of-words baseline. It maps each token to a 128-dimensional embedding, then averages all token embeddings in a review using `GlobalAveragePooling1D`. This average vector is the only input to the classifier – word order is completely discarded. A `SpatialDropout1D(0.2)` layer after the embedding helps prevent co-adaptation of embedding dimensions. The averaged representation is fed through `Dense(64, ReLU)` with L2 regularization (10^{-4}), then `Dropout(0.4)`, and finally a sigmoid output.

The model has 2,568,321 trainable parameters and was compiled with Adam ($\text{lr} = 10^{-3}$) and binary cross-entropy. Training ran 13 out of a maximum 20 epochs with batch size 64. `ReduceLROnPlateau` halved the learning rate whenever validation loss stalled for one epoch, and early stopping (patience 3) restored the weights from epoch 10, which had the best validation loss of 0.3051. Despite having no access to word order, the MLP reached 88.64% validation accuracy. This tells us that many IMDb reviews contain enough strongly polar words (e.g. “terrible”, “brilliant”) for a simple average to work reasonably well.

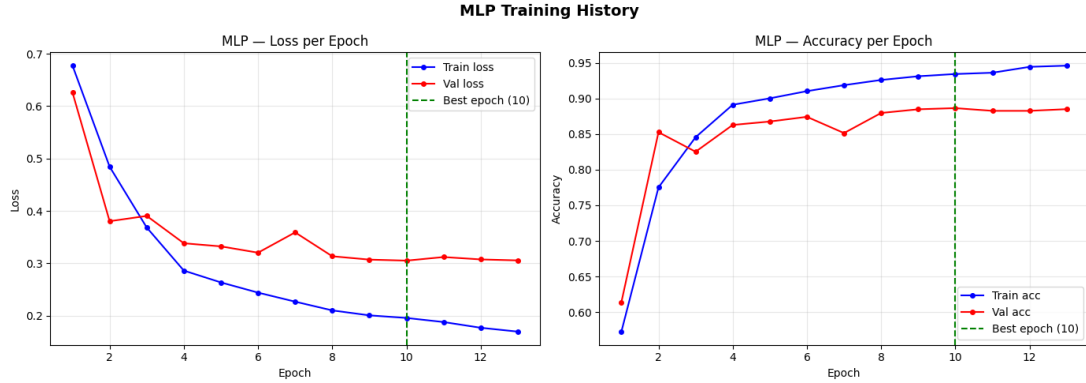


Figure 2: MLP training curves across 13 epochs. Training accuracy keeps rising while validation accuracy plateaus after epoch 8–9, showing mild overfitting.

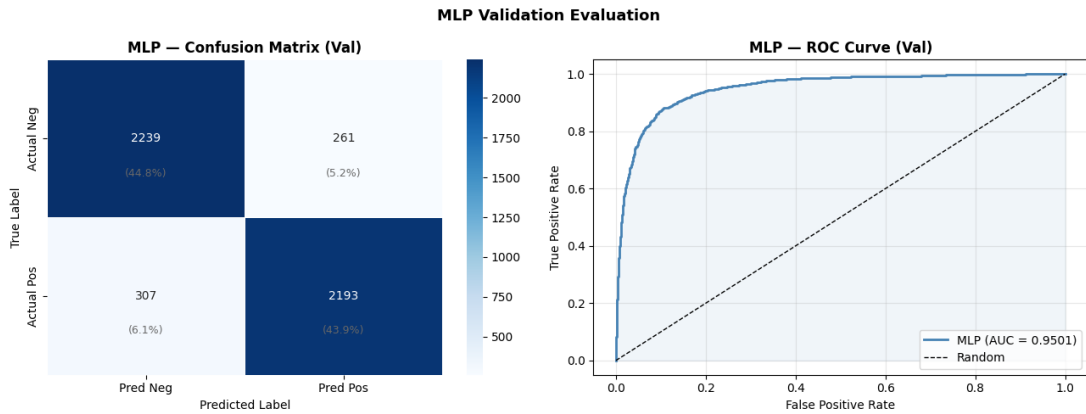


Figure 3: MLP validation confusion matrix and ROC curve (AUC = 0.9501).

B. 1D Convolutional Neural Network (1D-CNN)

The CNN follows the multi-kernel design from Kim (2014) [2]. After the same $20,000 \times 128$ embedding layer and a `SpatialDropout1D(0.3)`, three parallel `Conv1D` branches with kernel sizes 3, 4, and 5 scan the token sequence. Each branch has 64 filters and uses ReLU activation, so the three kernels act as bigram, trigram, and four-gram detectors. `GlobalMaxPooling1D` on each branch picks out the strongest activation regardless of position, and the three 64-dimensional outputs are concatenated into a 192-dimensional feature vector. This goes through `Dropout(0.4)`, `Dense(64, ReLU, L2=1e-4)`, `Dropout(0.3)`, and a sigmoid output.

Total trainable parameters: 2,670,913. Optimizer: Adam at $\text{lr} = 10^{-3}$. Training stopped at epoch 6 out of 20 (early stopping patience 3, best epoch was epoch 3 with `val_loss` = 0.2969). One clear pattern in the training curves is that training accuracy jumped to about 99% by epoch 6 while validation accuracy stayed around 88%. This gap shows overfitting past epoch 3, which is exactly why early stopping is necessary. The CNN’s validation accuracy (88.50%) and AUC (0.9545) are close to the MLP’s, but the slightly higher AUC suggests that the convolution kernels extract some useful local phrase patterns that bag-of-words averaging misses.



Figure 4: 1D-CNN training curves and learning-rate schedule. Note the sharp train-val divergence after epoch 3.

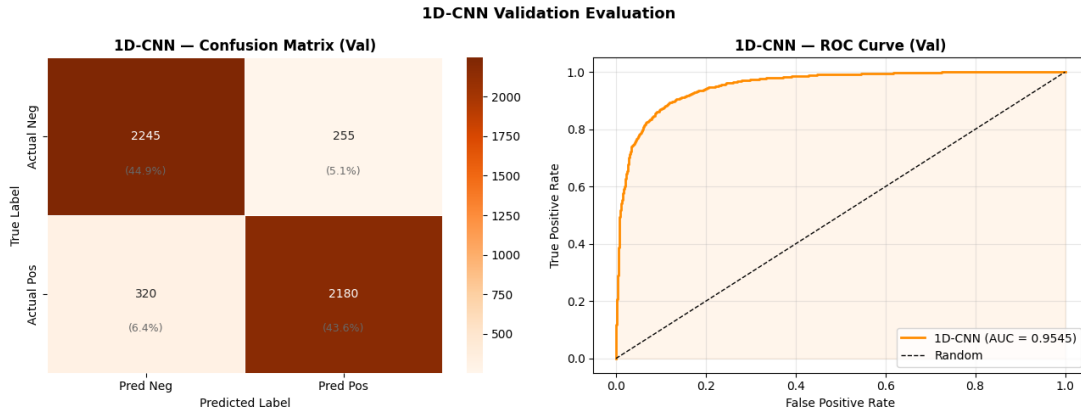


Figure 5: 1D-CNN validation confusion matrix and ROC curve (AUC = 0.9545).

C. Bidirectional LSTM (BiLSTM)

The BiLSTM reads the token sequence in both forward and backward directions. After the embedding and `SpatialDropout1D(0.3)`, a single `Bidirectional(LSTM(64))` layer processes the full 512-token sequence. We set `mask_zero=True` on the embedding so the LSTM ignores padding tokens, `dropout=0.3` on the input-to-hidden connections, and `recurrent_dropout=0.2` on the hidden-to-hidden connections. The resulting 128-dimensional output (64 forward + 64 backward) is classified through `Dense(64, ReLU, L2=1e-4)`, `Dropout(0.4)`, and sigmoid.

Total parameters: 2,667,137. Optimizer: Adam at $lr = 5 \times 10^{-4}$ (lower than MLP/CNN because recurrent models tend to be more sensitive to large learning rates). The BiLSTM was the slowest model to train – roughly 11 minutes per epoch on CPU in Colab, versus under a minute for MLP and CNN. Early stopping triggered after 5 epochs and rolled back to epoch 2 (`val_loss = 0.3143`). Validation accuracy came in at 87.12% with AUC 0.9404, the lowest among all four models. The training curves show that training loss kept dropping while validation loss stalled after epoch 2, pointing to overfitting. We suspect the combination of long sequences (512 tokens), limited training data (20k samples), and the large number of recurrent parameters makes the BiLSTM harder to regularize properly.



Figure 6: BiLSTM training curves. Validation loss stops improving after epoch 2; the learning rate is halved multiple times but does not help.

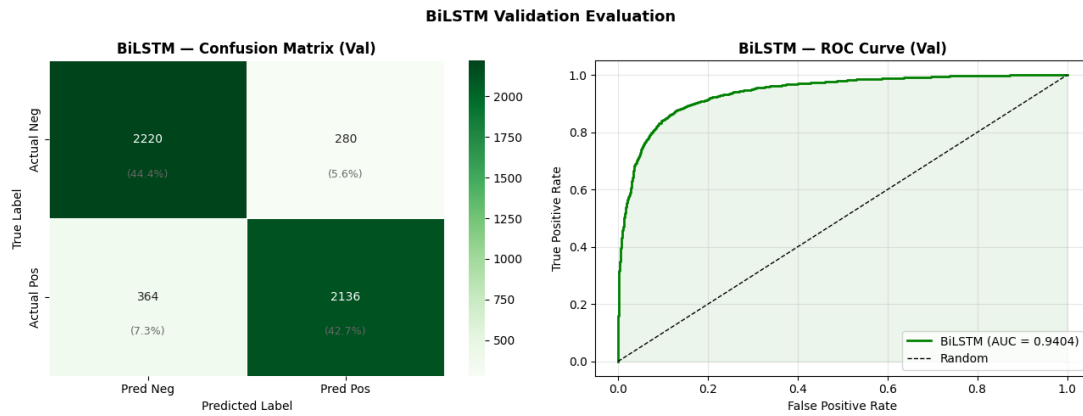


Figure 7: BiLSTM validation confusion matrix and ROC curve (AUC = 0.9404).

D. DistilBERT (Fine-tuned Pretrained Transformer)

DistilBERT starts from a pretrained checkpoint (distilbert-base-uncased) that already encodes knowledge of English syntax and semantics from a large pretraining corpus. We add a single-output classification head (num_labels=1) and fine-tune all 66,954,241 parameters end-to-end. The optimizer is AdamW (lr = 2×10^{-5} , weight decay = 0.01) with a linear warmup over the first 10% of training steps (625 out of 6,250 total) followed by linear decay. Gradients are clipped at max norm 1.0 to prevent spikes during early fine-tuning. The loss function is BCEWithLogitsLoss, and predictions are obtained by applying sigmoid to the raw logit.

Because DistilBERT has $25\times$ more parameters than the other three models, we used a batch size of 16 (vs. 64 for the others) and trained on a T4 GPU instead of CPU. Even so, training was fast – about 5 minutes per epoch. Early stopping (patience 2) triggered at epoch 4 and rolled back to epoch 2, which had val_loss = 0.2655 and validation accuracy = 90.72%. This is about 2 percentage points above the MLP and CNN, and 3.6 points above the BiLSTM. The pretrained representations give DistilBERT a clear head start: instead of learning all sentiment features from scratch on just 20,000 reviews, it only needs to adjust representations that already capture word meaning and context.

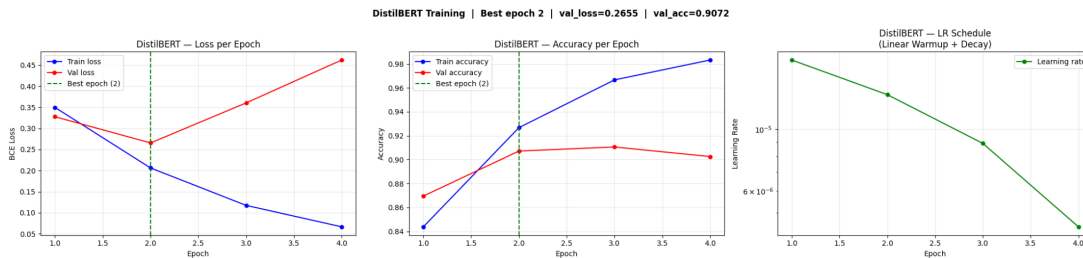


Figure 8: DistilBERT training curves with linear warmup/decay schedule. The model reaches peak validation accuracy at epoch 2.

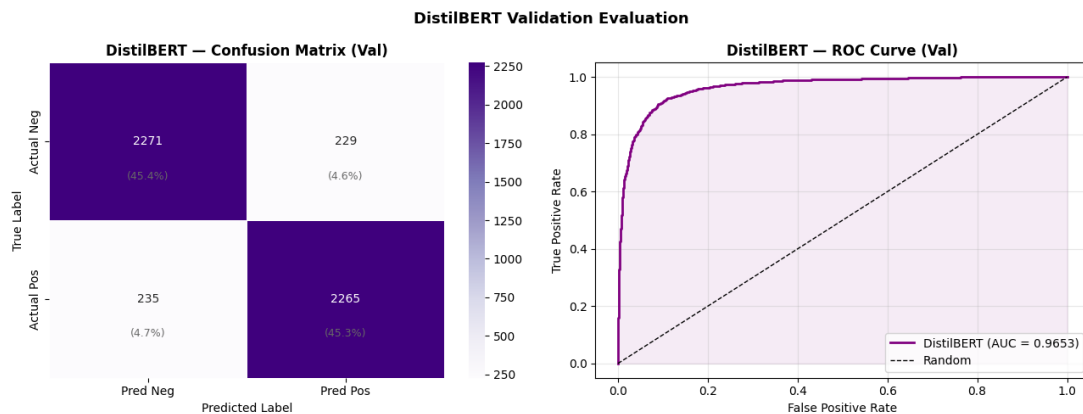


Figure 9: DistilBERT validation confusion matrix and ROC curve (AUC = 0.9653).

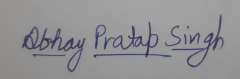
TABLE 2. Models Implemented — Status and Preliminary Validation Results

Model	Owner (Member)	Status	Preliminary Metric	Notes
MLP	Y Kedarnath Chowdary	Complete	Val Acc: 88.64% Val AUC: 0.9501	13 epochs, early stop at epoch 10; baseline without sequence modelling
1D-CNN	Abhay Pratap Singh	Complete	Val Acc: 88.50% Val AUC: 0.9545	6 epochs, early stop at epoch 3; multi-kernel (3,4,5) architecture
BiLSTM	Rallapalli Dheeraj C.	Complete	Val Acc: 87.12% Val AUC: 0.9404	5 epochs, early stop at epoch 2; bidirectional recurrent model
DistilBERT	D. Vishwatej	Complete	Val Acc: 90.72% Val AUC: 0.9653	4 epochs, early stop at epoch 2; fine-tuned pretrained Transformer

4. INDIVIDUAL CONTRIBUTION LOG (TO DATE)

Each team member was responsible for one model end-to-end: writing the notebook, debugging training issues, producing evaluation plots, and saving artefacts to the shared Google Drive. The preprocessing code was written by Y Kedarnath Chowdary as part of the MLP notebook and then reused by the CNN and BiLSTM notebooks through the shared tokenizer file. D. Vishwatej set up the GitHub repository and coordinated the report writing. Table III logs the specific tasks completed by each member up to this point.

TABLE 3. Individual Contribution Log

Member Name	Reg. No.	Task(s) Completed	Signature
D. Vishwatej	230953220	Built and trained the DistilBERT notebook; set up AdamW with linear warmup schedule; ran validation evaluation (90.72% acc); set up GitHub repo; coordinated report writing and compiled the interim L ^A T _E X document	
Abhay Pratap Singh	230911522	Built and trained the 1D-CNN notebook with three parallel kernel branches (3, 4, 5); tuned dropout and early-stopping settings; ran validation evaluation (88.50% acc); generated confusion matrix and ROC plots	
Rallapalli Dheeraj C.	230911530	Built and trained the BiLSTM notebook; configured bidirectional LSTM with input and recurrent dropout; ran validation evaluation (87.12% acc); produced training-curve and evaluation plots; investigated slow convergence	
Y Kedarnath Chowdary	230911176	Built and trained the MLP baseline notebook; wrote the shared tokenizer fitting code; ran the review-length analysis (MAX_LEN=512, 92.3% coverage); ran validation evaluation (88.64% acc); saved shared artefacts to Drive	

5. RISK / PLAN FOR REMAINING WORK

Remaining Work

All four models are trained and validated. What remains is the final evaluation and analysis for the Part C report:

- Final test-set evaluation:** Run all four models on the held-out 25,000-review test set. Record accuracy, precision, recall, F1-score, and AUC under the same protocol.
- Comparative analysis:** Put all four models in a single results table and discuss why their numbers differ. The discussion matters more than the numbers themselves – the grading rubric weighs analysis over raw performance.
- Error analysis:** Look at misclassified reviews for at least two models (CNN and BiLSTM). Check whether errors cluster around certain confidence ranges, review lengths, or linguistic patterns like sarcasm and negation.
- Final report:** Write all Part C sections – Abstract, Introduction, Methodology (one subsection per model, written by the member who built it), Experimental Setup, Results and Discussion, Conclusion, Future Work, and References (minimum 12).

Expected Challenges

- **BiLSTM generalisation gap:** The BiLSTM already has the lowest validation accuracy (87.12%) and AUC (0.9404). There is a real chance the gap widens further on the test set. If it does, we plan to discuss possible causes – over-fitting to sequence artefacts, sensitivity to the learning rate, or the high ratio of recurrent parameters to training examples – but we will not re-tune or cherry-pick results after seeing test numbers.
- **Sequence-length mismatch:** The three word-level models use `MAX_LEN=512`, but DistilBERT uses 256 subword tokens because of GPU memory limits on the T4. Since subwords and words are not one-to-one, the effective text coverage may differ. We need to acknowledge this when comparing results rather than ignore it.
- **Tokenisation differences:** Word-level and WordPiece tokenization handle unknown words, compound words, and morphology differently. This makes a perfectly apples-to-apples comparison impossible, but it also reflects the practical trade-off between training from scratch and fine-tuning a pretrained model.
- **Report length:** The Part C template asks for a dense IEEE-style write-up. Fitting all four model descriptions, a proper discussion section, error analysis, and 12+ references into a coherent document will take time.

Timeline to Final Submission

Sl. No.	Component / Work Element / Milestone	Expected Completion
1	Final test-set evaluation for all four models	5 Oct 2026
2	Comparative analysis: results table, confusion matrices, ROC curves	14 Oct 2026
3	Error analysis on misclassified examples (CNN, BiLSTM)	18 Oct 2026
4	Report writing: complete Part C (all sections)	27 Oct 2026
5	Presentation slides, viva preparation, final code push to GitHub	31 Oct 2026

REFERENCES

- [1] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, “Learning Word Vectors for Sentiment Analysis,” in *Proc. 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, 2011, pp. 142–150. [Online]. Available: <https://aclanthology.org/P11-1015/>
- [2] Y. Kim, “Convolutional Neural Networks for Sentence Classification,” in *Proc. 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1746–1751. [Online]. Available: <https://aclanthology.org/D14-1181/>
- [3] N. M. Ali, M. M. M. Abd El Hamid, and A. Youssif, “Sentiment Analysis for Movies Reviews Dataset Using Deep Learning Models,” *International Journal of Data Mining & Knowledge Management Process*, vol. 9, no. 2/3, pp. 19–27, 2019. [Online]. Available: <https://doi.org/10.5121/ijdkp.2019.9302>
- [4] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: <https://doi.org/10.1162/neco.1997.9.8.1735>
- [5] M. Schuster and K. K. Paliwal, “Bidirectional Recurrent Neural Networks,” *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997. [Online]. Available: <https://doi.org/10.1109/78.650093>
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, vol. 1, 2019, pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423/>
- [7] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter,” in *Proc. 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing, NeurIPS*, 2019. [Online]. Available: <https://arxiv.org/abs/1910.01108>
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems 30*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [9] Z. Yang, D. Yang, C. Dyer, X. He, A. Smola, and E. Hovy, “Hierarchical Attention Networks for Document Classification,” in *Proc. NAACL-HLT*, 2016, pp. 1480–1489. [Online]. Available: <https://aclanthology.org/N16-1174/>
- [10] J. Howard and S. Ruder, “Universal Language Model Fine-tuning for Text Classification,” in *Proc. 56th Annual Meeting of the Association for Computational Linguistics*, vol. 1, 2018, pp. 328–339. [Online]. Available: <https://aclanthology.org/P18-1031/>